

Poster: Design of a Digital Forensics Framework for Spacecraft Flight Software

Logan Flowers
New Mexico State University
loganf@nmsu.edu

Efrén López-Morales 
New Mexico State University
elopezm@nmsu.edu

Abstract—Modern spacecraft flight software (FS) provides limited support for collecting and preserving digital evidence following cybersecurity incidents. This work presents the design of an onboard FS forensics framework that continuously collects security-relevant artifacts and transforms them into structured forensic records for incident reconstruction. The proposed architecture includes components for evidence collection, validation, normalization, timeline generation, prioritization, and onboard storage prior to downlink. We implement an initial prototype using NASA’s core Flight System (cFS) within the OpenSatKit environment. Preliminary results demonstrate the feasibility of integrating forensic collection into cFS and acquiring security-relevant artifacts from core flight software services during controlled command experiments.

Index Terms—satellite security, digital forensics, NASA cFS, flight software

I. INTRODUCTION

Modern spacecraft are increasingly software-driven and interconnected, expanding their exposure to cyber threats. Recent research has demonstrated exploitable vulnerabilities in real satellite firmware [1], while satellite honeypots have captured real adversarial interactions targeting spacecraft [2]. These findings highlight the need not only to prevent and detect attacks, but also to determine *what happened after a security incident*. Digital forensics can support this process by preserving evidence needed to reconstruct malicious activity.

Prior work has begun to characterize the sources of digital evidence available in spacecraft systems [3]. However, mechanisms for systematically collecting this evidence from onboard flight software remain limited. Spacecraft flight software (FS) already produces valuable forensic artifacts, including telemetry, and inter-application messages. However, these artifacts are primarily generated for spacecraft operation and monitoring rather than forensic analysis.

In this work, we present the initial design of an onboard digital forensics framework for spacecraft flight software. The framework collects security-relevant observations and transforms them into normalized forensic records that can be prioritized, correlated into incident timelines, stored onboard, and later downlinked for analysis. We prototype the approach using NASA’s core Flight System (cFS) and demonstrate the feasibility of collecting forensic artifacts from cFS services during controlled command experiments.

II. BACKGROUND

Modern spacecraft use reusable FS frameworks that provide common services for command handling, telemetry, and application execution. Examples include NASA’s core Flight System (cFS) [4], NASA’s F Prime [5], and the Space and Planetary Exploration Laboratory’s SUCHAI flight software [6]. These frameworks differ in their architectures and mechanisms for exposing spacecraft activity, motivating a forensic architecture that can adapt its data collection while maintaining a common representation of forensic evidence.

NASA’s core Flight System (cFS) [4] is a modular flight software architecture built around the core Flight Executive (cFE) [7]. cFE provides common services for communication, event reporting, timing, and application execution. In particular, the Software Bus carries messages between applications, while Event Services and housekeeping telemetry expose information about application activity and spacecraft state.

These mechanisms produce artifacts that can be valuable for digital forensics. Commands, software bus messages, and timestamps can provide evidence of *what occurred*, *when it occurred*, and *which component was involved* in a security incident.

III. DESIGN

Overview. We design the forensic framework as an onboard application that continuously collects security-relevant flight software artifacts, including telemetry, commands, system state, and inter-application communication. As shown in Figure 1, these observations are validated, normalized, correlated, and prioritized to produce forensic records that can be stored onboard and later downlinked for analysis. The architecture separates FS-specific data collection from forensic processing to support adaptation to different flight software frameworks.

Interfacing Layer. The Interfacing Layer connects the framework to flight-software-specific communication mechanisms and translates collected observations into a common internal representation. Supporting another FS framework requires implementing an appropriate interface adapter while leaving the remaining forensic pipeline unchanged.

Collection and Handling. Collected observations are validated for completeness and required metadata, associated with their source, and normalized into a common forensic schema. Normalization standardizes fields such as timestamps, event types, and framework-specific representations.

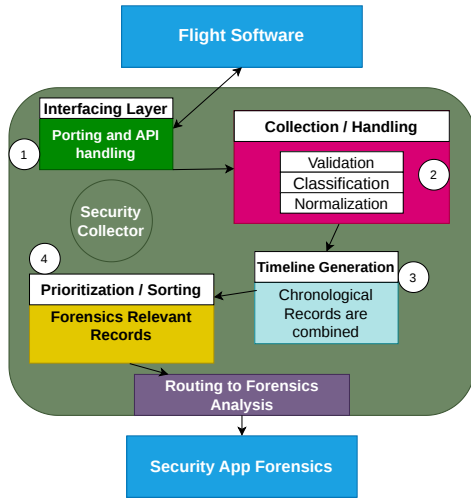


Fig. 1. Forensic collection and processing architecture.

Timeline Generation. Normalized observations are correlated using timestamps, source applications, command relationships, and other identifiers to reconstruct the sequence of activity surrounding a potential security incident.

Evidence Prioritization. Forensic records are assigned a priority based on their security relevance and severity. Prioritization determines which evidence should be preserved or transmitted first when onboard storage or communication bandwidth is constrained.

Storage. Generated forensic records are retained using the flight software’s onboard storage until they can be downlinked for ground-based analysis.

Together, these components provide a modular pipeline for transforming operational data into structured forensic evidence suitable for incident reconstruction.

IV. PRELIMINARY RESULTS

Testing Environment. We developed a Docker-based test environment using OpenSatKit (OSK) [8] to simulate a spacecraft running cFS. A prototype Security App was integrated with cFS to collect observations from cFE services. A separate Kali Linux attack environment communicates with the simulated spacecraft through the KIT-CI command interface and monitors telemetry through KIT-TO, enabling controlled attack experiments while observing the resulting cFS activity.

Attack Experiments. We developed scripts that generate CCSDS command packets and transmit them to cFS through KIT-CI. We first issued no-operation commands to validate communication and then generated abnormal commands containing invalid message identifiers, invalid checksums, unsupported functions, and incorrect data lengths. A second script captures CCSDS telemetry transmitted through KIT-TO, allowing spacecraft responses to the commands to be observed.

Forensic Data Acquisition. We initially collected evidence from housekeeping telemetry and Event Services. Housekeeping telemetry provides recurring information about the

state of cFS applications, while generated events identify the originating application, event identifier, and observed activity.

Our preliminary results show that these sources provide information identifying the source, time, and type of spacecraft activity, demonstrating their usefulness for constructing normalized forensic records.

V. FUTURE WORK

Based on our preliminary results, we will complete the integration of the Security App with cFS and implement the remaining components of the forensic pipeline, including normalization, evidence prioritization, timeline generation, and onboard storage.

We will expand the attack experiments beyond malformed CCSDS commands to scenarios that modify spacecraft state or application behavior. The resulting forensic artifacts will be collected and correlated to reconstruct each incident, allowing us to evaluate whether the framework can identify the sequence of events, affected components, and actions leading to a security-relevant state. We will also measure the computational, storage, and communication overhead introduced by continuous forensic collection.

Finally, we will evaluate the portability of the framework by implementing an interface adapter for a second flight software platform, such as NASA F Prime or SUCHAI. This evaluation will determine which forensic abstractions and processing components can be reused across flight software frameworks and which require platform-specific adaptation.

ACKNOWLEDGMENT

This work was supported in part by startup funds from New Mexico State University and the College of Arts & Sciences.

REFERENCES

- [1] J. Willbold, M. Schloegel, M. Vögele, M. Gerhardt, T. Holz, and A. Abbasi, “Space odyssey: An experimental software security analysis of satellites,” in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1–19.
- [2] E. López-Morales, U. Planta, G. Marra, C. González, J. Hopkins, M. Garosi, E. Obreque, C. Rubio-Medrano, and A. Abbasi, “Honeysat: A network-based satellite honeypot framework,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2026.
- [3] O. Ramsey and E. López-Morales, “Poster: Mapping the evidence: A taxonomy for satellite digital forensics,” in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, May 2026.
- [4] D. McComas, J. Wilmot, and A. Cudmore, “The core flight system (cfs) community: Providing low cost solutions for small spacecraft,” in *Annual AIAA/USU conference on small satellites*, no. GSFC-E-DAA-TN33786, 2016.
- [5] R. L. Bocchino Jr., T. K. Canham, G. J. Watney, L. J. Reder, and J. W. Levison, “F prime: An open-source framework for small-scale flight software systems,” in *Proceedings of the 32nd Annual AIAA/USU Conference on Small Satellites*, no. SSC18-XII-04, 2018.
- [6] C. E. Gonzalez, C. J. Rojas, A. Bergel, and M. A. Diaz, “An architecture-tracking approach to evaluate a modular and extensible flight software for cubesat nanosatellites,” *IEEE Access*, 2019.
- [7] *Core Flight Executive Users Guide*, National Aeronautics and Space Administration.
- [8] D. McComas, “Opensatkit: A flight software system educational platform,” in *Proceedings of the 35th AIAA/USU Conference on Small Satellites*, 2021. [Online]. Available: <https://github.com/OpenSatKit>